

Experiment-10

AIM: Simulate smoothing and sharpening operations on images using frequency domain filters.

Theory:

Smoothing and sharpening operations can also be performed in the frequency domain using filters such as the Gaussian filter and the Laplacian filter.

Smoothing in Frequency Domain:

To perform smoothing in the frequency domain, we apply a low-pass filter that attenuates high-frequency components. This helps to blur the image and reduce noise. One common approach is to use a Gaussian filter in the frequency domain.

Sharpening in Frequency Domain:

To perform sharpening in the frequency domain, we apply a high-pass filter that enhances high-frequency components. This amplifies the edges and fine details in the image. One common approach is to use a combination of a high-pass filter and the original image.

Program

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def apply_gaussian_filter(image, sigma):
    # Convert image to float32 for Fourier Transform
    image = np.float32(image)

    # Perform Fourier Transform
    frequency_domain = cv2.dft(image, flags=cv2.DFT_COMPLEX_OUTPUT)

    # Shift the zero-frequency component to the center of the spectrum
    shifted_frequency_domain = np.fft.fftshift(frequency_domain)

    # Create Gaussian filter mask
    rows, cols = image.shape
    crow, ccol = rows // 2, cols // 2
    mask = np.zeros((rows, cols, 2), np.float32)
    for i in range(rows):
        for j in range(cols):
            mask[i, j] = np.exp(-((i - crow) ** 2 + (j - ccol) ** 2) / (2 * sigma ** 2))

    # Apply the Gaussian filter in the frequency domain
    filtered_frequency_domain = shifted_frequency_domain * mask

    # Shift the zero-frequency component back to the corner
    shifted_filtered_frequency_domain = np.fft.fftshift(filtered_frequency_domain)

    # Perform Inverse Fourier Transform to obtain the filtered image
```



```
filtered_image = cv2.idft(shifted_filtered_frequency_domain, flags=cv2.DFT_SCALE |  
cv2.DFT_REAL_OUTPUT)
```

```
# Convert the filtered image back to uint8  
filtered_image = np.uint8(filtered_image)
```

```
return filtered_image
```

```
def apply_sharpening_filter(image, strength):
```

```
# Convert image to float32 for Fourier Transform  
image = np.float32(image)
```

```
# Perform Fourier Transform
```

```
frequency_domain = cv2.dft(image, flags=cv2.DFT_COMPLEX_OUTPUT)
```

```
# Shift the zero-frequency component to the center of the spectrum
```

```
shifted_frequency_domain = np.fft.fftshift(frequency_domain)
```

```
# Create high-pass filter mask
```

```
rows, cols = image.shape
```

```
crow, ccol = rows // 2, cols // 2
```

```
mask = np.zeros((rows, cols, 2), np.float32)
```

```
mask[crow - strength:crow + strength, ccol - strength:ccol + strength] = 1
```

```
# Apply the high-pass filter in the frequency domain
```

```
filtered_frequency_domain = shifted_frequency_domain * mask
```

```
# Shift the zero-frequency component back to the corner
```

```
shifted_filtered_frequency_domain = np.fft.fftshift(filtered_frequency_domain)
```

```
# Perform Inverse Fourier Transform to obtain the filtered image
```

```
filtered_image = cv2.idft(shifted_filtered_frequency_domain, flags=cv2.DFT_SCALE |  
cv2.DFT_REAL_OUTPUT)
```

```
# Convert the filtered image back to uint8
```

```
filtered_image = np.uint8(filtered_image)
```

```
return filtered_image
```

```
# Load the input image
```

```
image_path = '/content/Certificate 1698299913.jpg'
```

```
input_image = cv2.imread(image_path, 0) # Load the image in grayscale
```

```
# Apply Gaussian filter
```

```
sigma = 20 # Adjust the value to control the amount of smoothing
```

```
smoothed_image = apply_gaussian_filter(input_image, sigma)
```

```
# Apply sharpening filter
```

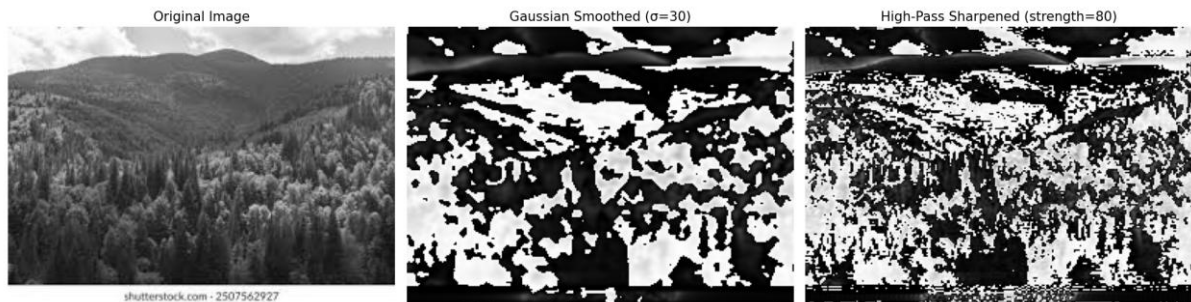
```
strength = 20 # Adjust the value to control the strength of sharpening
```

```
sharpened_image = apply_sharpening_filter(input_image, strength)
```

```
# Display the original image and the filtered images side by side
combined_image = np.hstack((input_image, smoothed_image, sharpened_image))
plt.imshow(combined_image, cmap='gray')
plt.title("Original | Smoothed | Sharpened")
plt.axis('off')
plt.show()

# Save the filtered images (optional)
smoothed_path = 'smoothed_image.jpg'
sharpened_path = 'sharpened_image.jpg'
cv2.imwrite(smoothed_path, smoothed_image)
cv2.imwrite(sharpened_path, sharpened_image)
print(f"Smoothed image saved at: {smoothed_path}")
print(f"Sharpened image saved at: {sharpened_path}")
```

Output



Conclusion

- Frequency domain filtering successfully applied using Fourier Transform.
- Gaussian filter smoothed the image by attenuating high frequencies (removes noise and details).
- High-pass filter sharpened the image by preserving high frequencies (enhances edges).
- Frequency domain approach provides precise control over specific frequency components for filtering.



Marwadi University
Faculty of Engineering and Technology
Department of Information and Communication Technology